



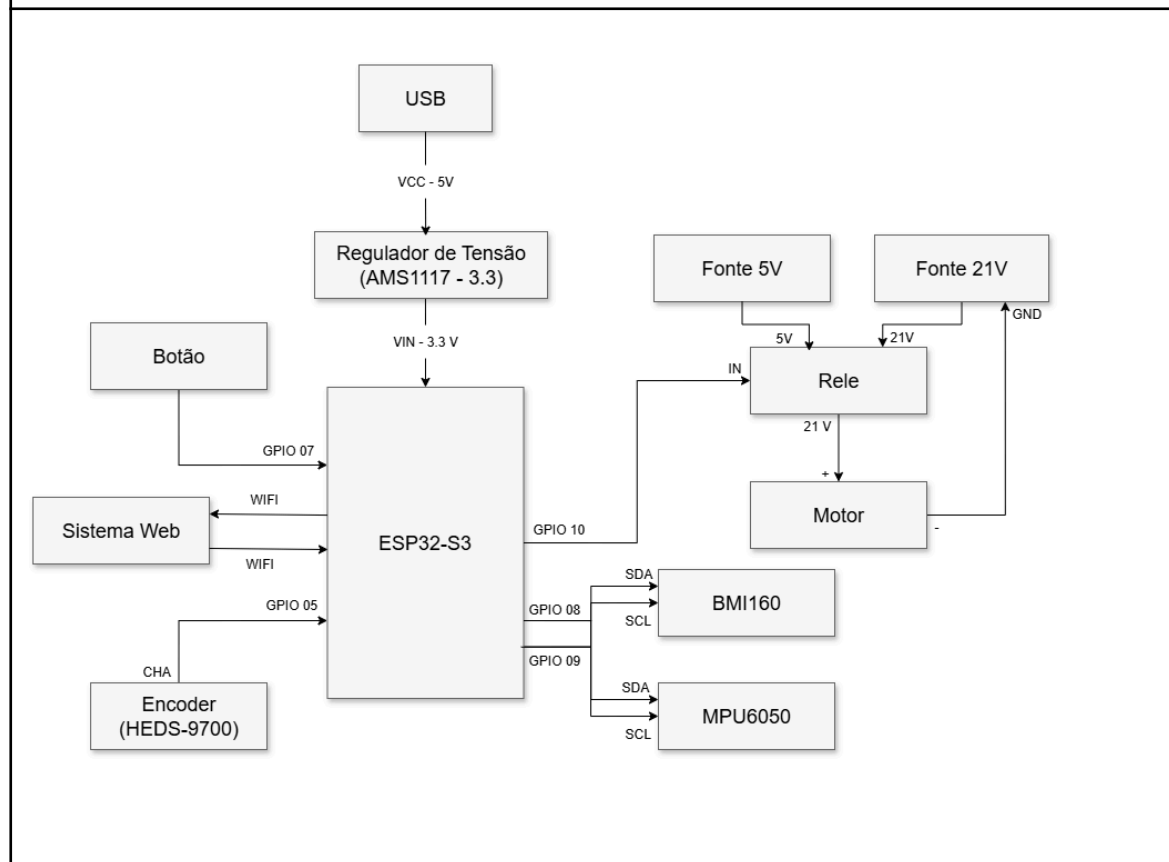
INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA DA
PARAÍBA
CAMPUS CAMPINA GRANDE
BACHARELADO EM ENGENHARIA DA COMPUTAÇÃO

Equipe: Gabriel Domingos, Luiz Oliveira de Souza Neto, Ryan Soares Nunes
Jorvino

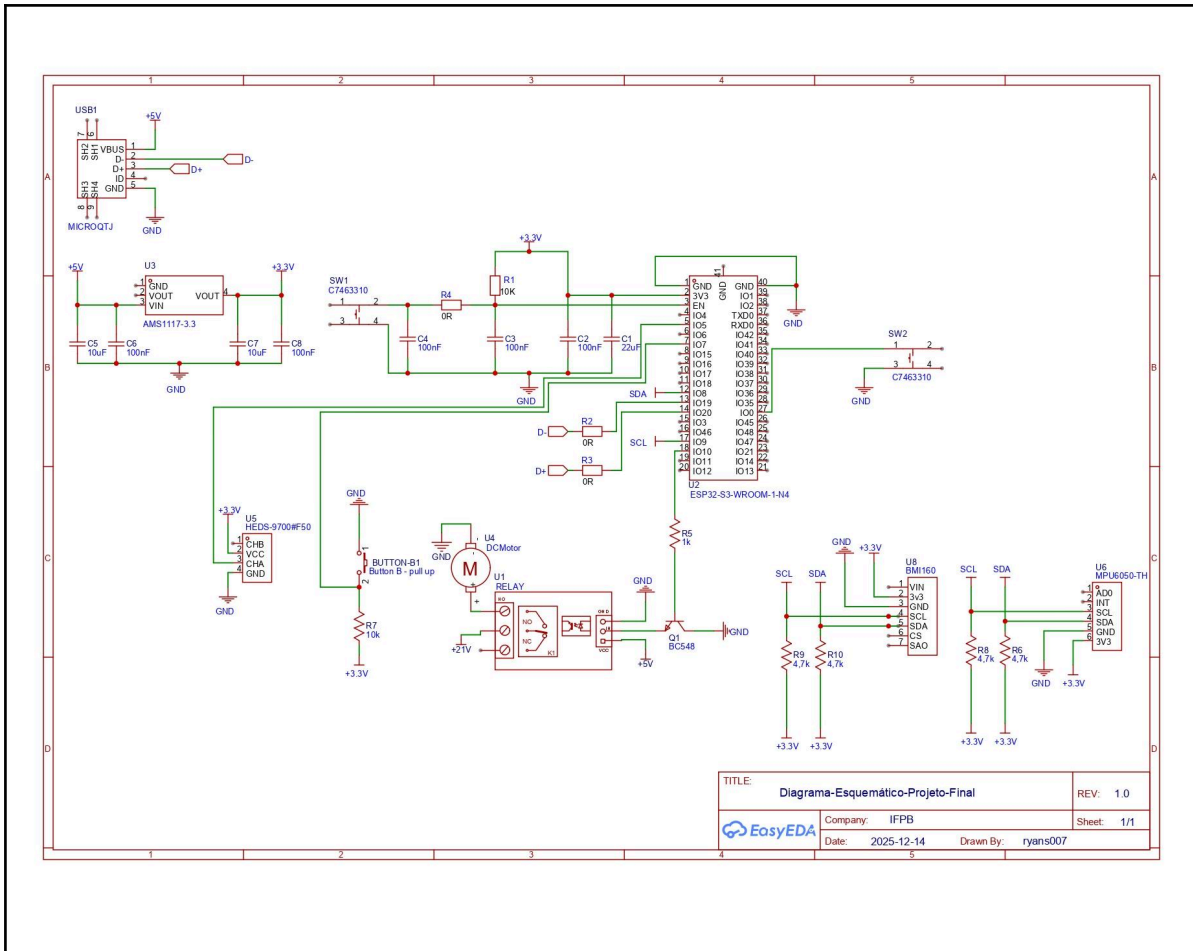
Campina Grande - PB

2025

Diagrama em blocos do sistema.



Esquemático.



Descrição das tarefas e fluxos FreeRTOS.

O firmware foi desenvolvido utilizando o sistema operacional de tempo real **FreeRTOS** sobre a estrutura do ESP-IDF. A aplicação é dividida em quatro tarefas principais que operam de forma concorrente, coordenadas por prioridades e semáforos (Mutex) para garantir a integridade dos dados e o acesso seguro aos periféricos compartilhados (como o barramento I2C).

Abaixo estão descritas as responsabilidades e o funcionamento de cada tarefa:

1. Task Sensor Monitor (Monitoramento de Hardware)

- **Prioridade:** 6 (Alta)
- **Periodicidade:** 500 ms (quando válido) a 1000 ms (quando inválido).
- **Descrição:** Esta tarefa atua como um *watchdog* de hardware para os sensores. Ela verifica periodicamente a integridade da conexão I2C.

- Se o sensor estiver desconectado, ela realiza uma varredura no barramento (scan) para identificar automaticamente se o dispositivo conectado é um MPU6050 ou um BMI160.
- Caso detecte uma desconexão, ela sinaliza o sistema (`g_sensor_is_valid = false`), impedindo que o motor seja acionado sem monitoramento, garantindo a segurança operacional.
- Utiliza o `xI2CMutex` para garantir acesso exclusivo ao barramento durante a verificação.

2. Task UI Control (Interface de Usuário)

- **Prioridade:** 5 (Média-Alta)
- **Periodicidade:** 50 ms.
- **Descrição:** Responsável pela interação direta com o operador via hardware (Botão e LEDs) e pelo acionamento do atuador (Relé/Motor).
 - Realiza a leitura do botão com tratamento de *debounce* (50ms).
 - Gerencia a lógica de segurança: o motor (Relé) só é ativado se houver uma solicitação do usuário **E** se a *Task Monitor* confirmar que o sensor está operacional.
 - Controla os LEDs de status (Verde para operação normal, Vermelho para falha detectada pela IA).

3. Task Processing (Coleta e Inteligência Artificial)

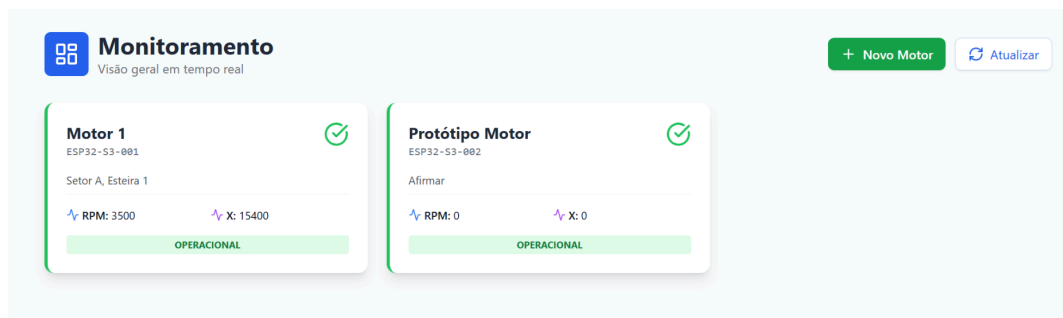
- **Prioridade:** 5 (Média-Alta)
- **Periodicidade:** 100 ms (10 Hz).
- **Descrição:** Esta é a tarefa central de processamento do sistema. Seu fluxo de execução segue as etapas:
 1. **Aquisição de Dados:** Bloqueia o `xI2CMutex` para ler os dados brutos de aceleração (eixos X, Y, Z) do sensor ativo.
 2. **Cálculo de Rotação:** Computa o RPM do motor com base na contagem de pulsos do encoder (capturados via interrupção de GPIO).
 3. **Inferência (TinyML):** Alimenta o modelo de *Machine Learning* embarcado com os dados de vibração e rotação.
 4. **Pós-Processamento (Votação):** Implementa um buffer de votação (janela de 10 amostras) para determinar a classe de falha predominante (Moda), evitando falsos positivos causados por ruídos momentâneos.
 5. **Atualização de Estado:** Ao final, bloqueia o `xDataMutex` para atualizar a estrutura global de dados (`g_latest_data`) que será consumida pela telemetria.

4. Task Telemetry (Conectividade e Envio de Dados)

- **Prioridade:** 4 (Média-Baixa)
- **Periodicidade:** 1000 ms (1 Hz).
- **Descrição:** Responsável por enviar as informações de monitoramento para o servidor remoto via protocolo HTTP.
 - Para não bloquear o processamento crítico (Task 3) durante o envio via Wi-Fi (que pode ser lento), esta tarefa realiza uma cópia rápida ("snapshot") dos dados globais protegidos pelo `xDataMutex`.

- Envia o pacote JSON contendo: vibração, RPM, status de falha e classificação da IA apenas se o motor estiver ligado e a conexão Wi-Fi ativa.

Capturas de tela do sistema web.





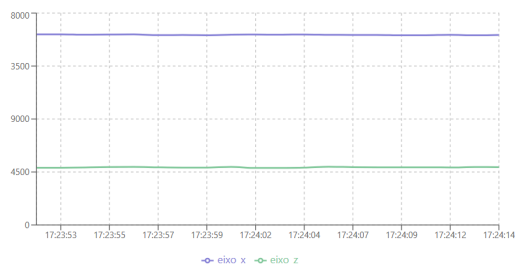
Protótipo Motor

ESP32-S3-002 - Assert

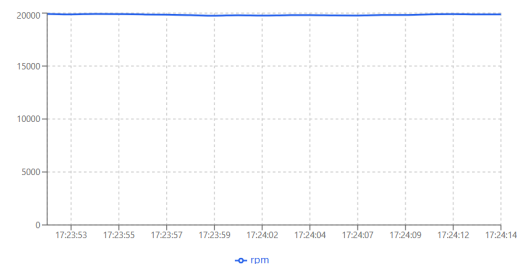
NORMAL



Vibração



Rotação (RPM)



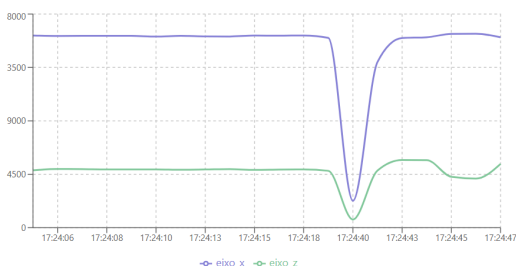
Protótipo Motor

ESP32-S3-002 - Assert

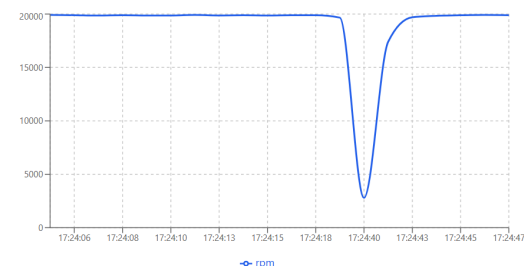
ALERTA (FALHA 1)



Vibração



Rotação (RPM)



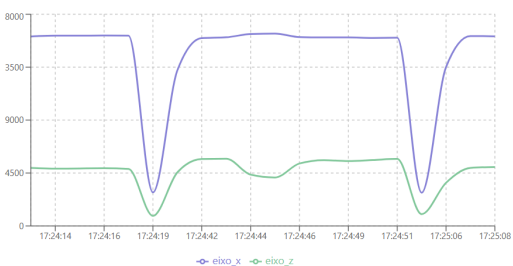
Protótipo Motor

ESP32-S3-002 - Assert

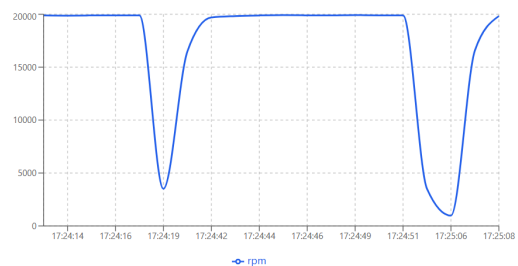
PERIGO (FALHA 2)



Vibração

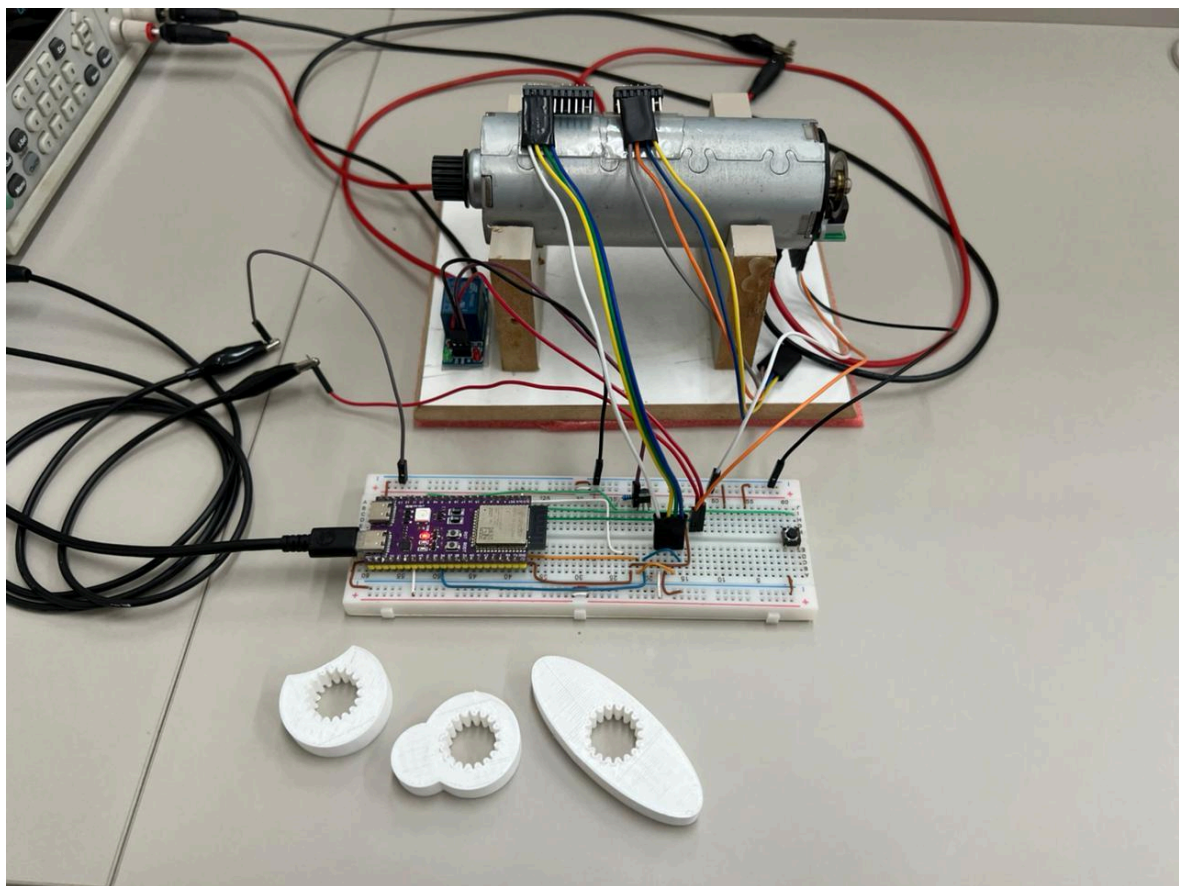


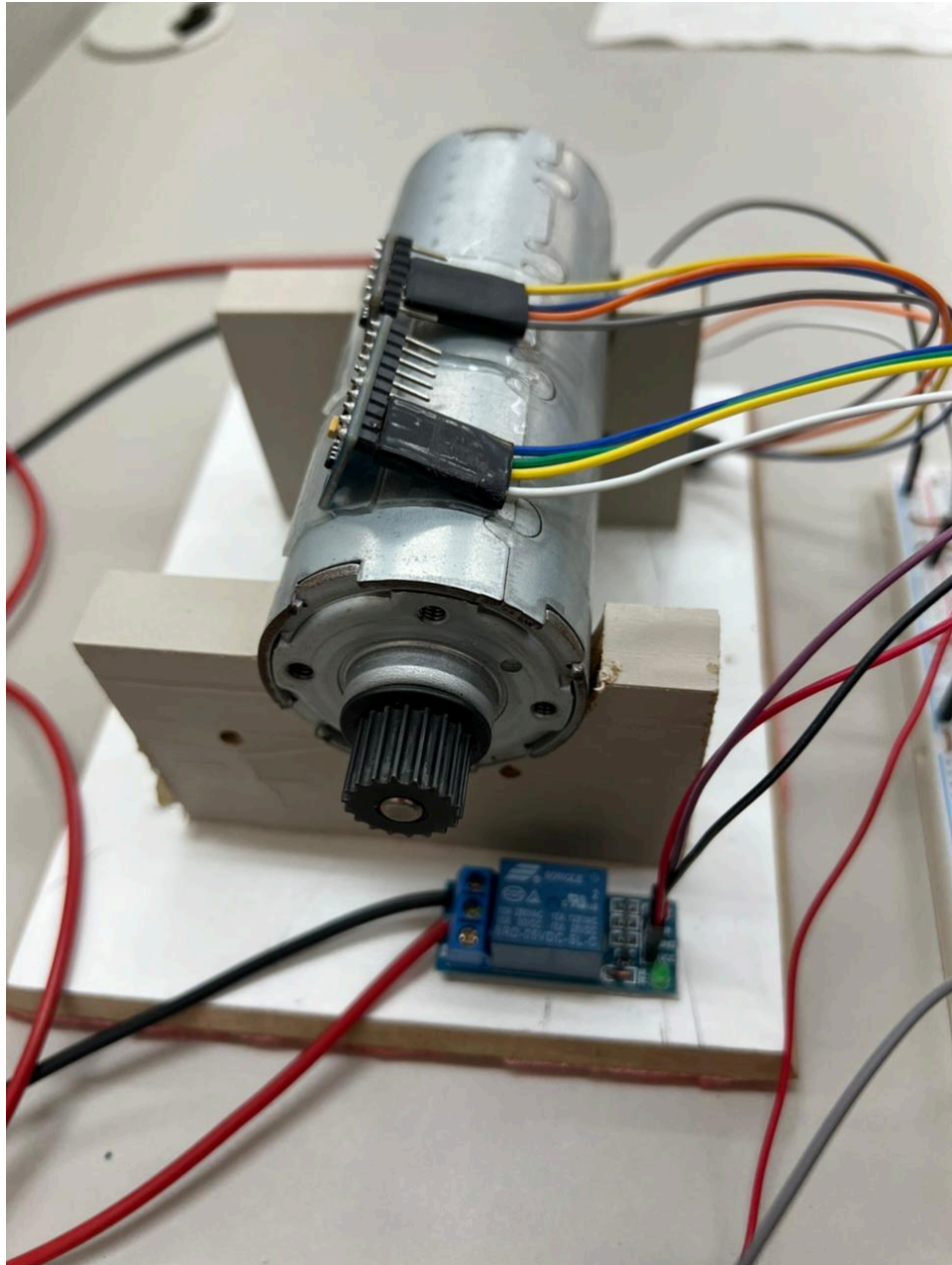
Rotação (RPM)

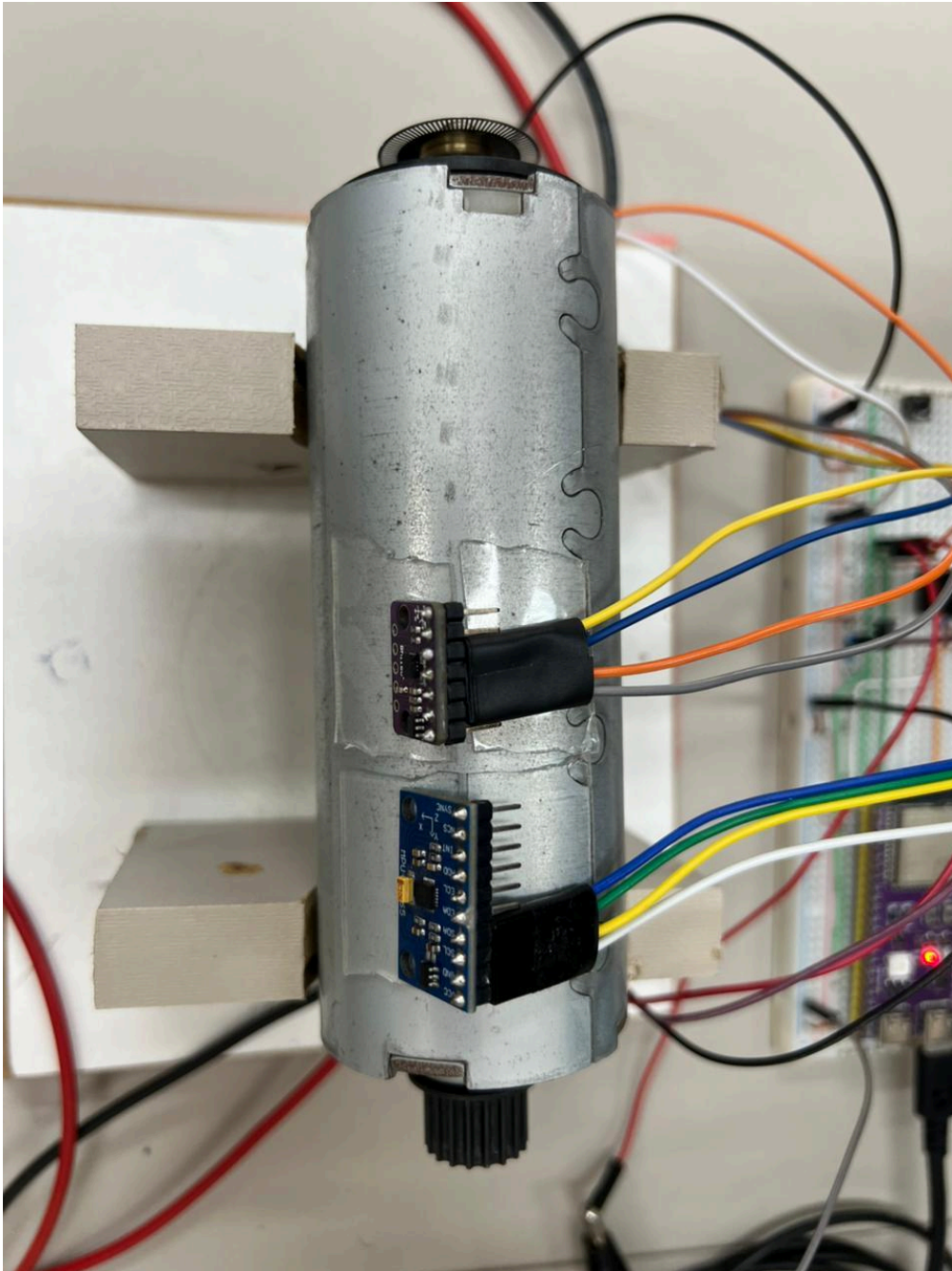


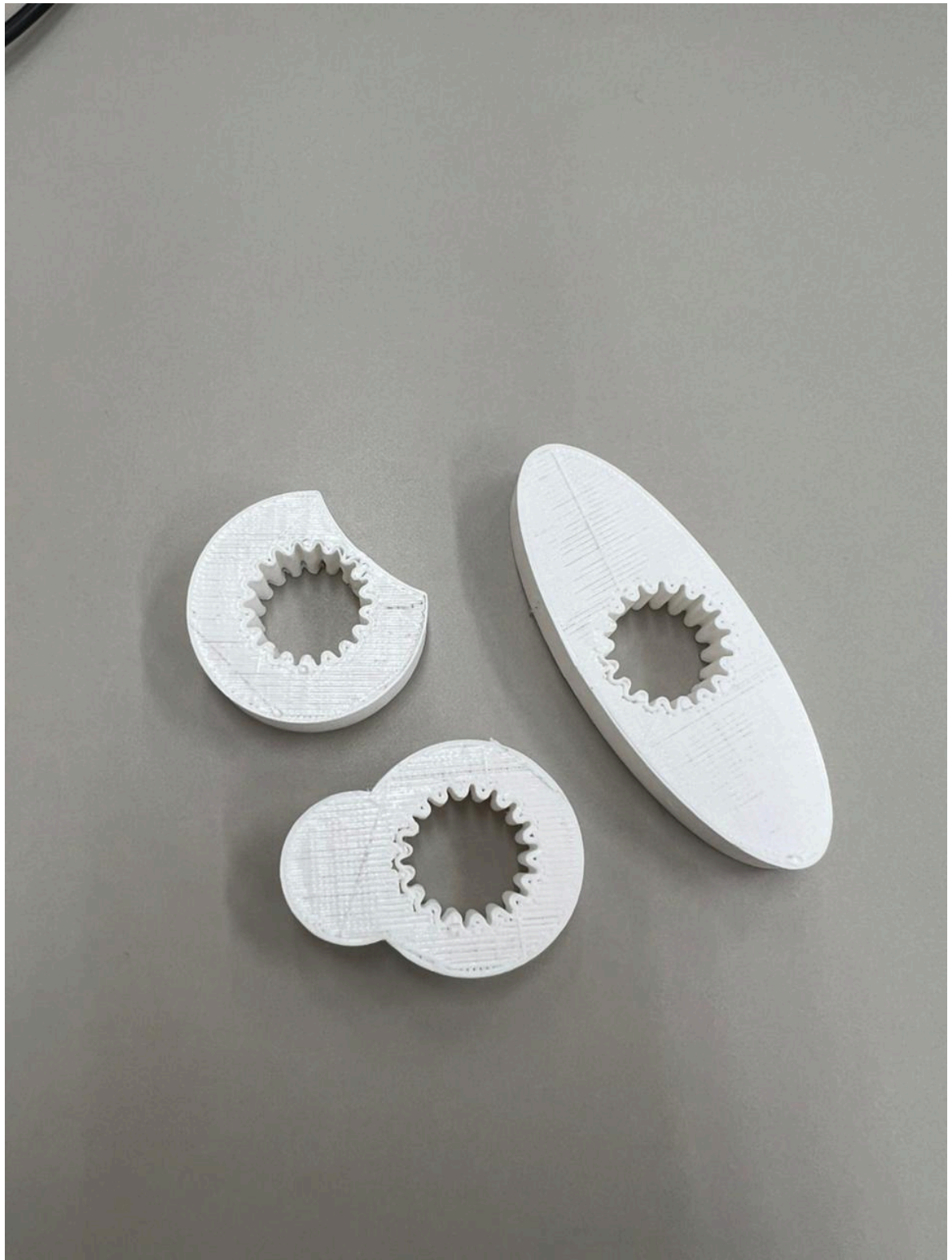


Fotos e vídeos do projeto em operação.









Link do vídeo no youtube mostrando o funcionamento do projeto:

<https://youtu.be/uesVgoCXy3U>

Dificuldades e soluções encontradas.

Durante o desenvolvimento do firmware para o sistema de monitoramento preditivo, a equipe enfrentou desafios técnicos significativos relacionados à integração de hardware e software em tempo real. Inicialmente, o processo de atualização do framework ESP-IDF resultou em inconsistências de compilação e referências indefinidas, exigindo uma reestruturação do sistema de build (CMake) e a limpeza completa de caches legados para garantir a correta vinculação das bibliotecas de Wi-Fi e armazenamento não volátil (NVS).

Um dos pontos críticos do projeto foi a arquitetura de software necessária para gerenciar tarefas concorrentes com requisitos temporais distintos. A solução adotada baseou-se no uso do sistema operacional de tempo real FreeRTOS, segmentando a aplicação em tarefas independentes para interface de usuário, monitoramento de sensores, processamento de dados e telemetria. Para garantir a integridade do barramento I2C, compartilhado entre sensores e tarefas de monitoramento, foi implementado um mecanismo de exclusão mútua (Mutex), prevenindo condições de corrida e travamentos do sistema.

Outro desafio relevante foi garantir a interoperabilidade entre dois sensores inerciais distintos, o MPU6050 e o BMI160, utilizando um único modelo de Inteligência Artificial. Para isso, além da implementação de rotinas para a identificação automática do sensor em tempo de execução via varredura de barramento, foi necessário desenvolver uma camada de calibração e normalização dos dados brutos. Esse processo assegurou que as leituras de ambos os sensores convergissem para a mesma escala e magnitude sob condições físicas idênticas, permitindo que a rede neural operasse com precisão independente do hardware conectado.

No aspecto de hardware, a leitura do encoder óptico apresentou instabilidade devido à interferência eletromagnética (EMI) gerada pelo acionamento do relé e transmissão Wi-Fi, o que ocasionava leituras espúrias de rotação (RPM) mesmo com o motor estático. O problema foi mitigado através da implementação de um filtro digital de debounce dentro da rotina de interrupção (ISR), que descarta pulsos com frequência fisicamente inverossímil para o sistema mecânico em questão.

Por fim, a integração com o servidor de telemetria exigiu o alinhamento rigoroso dos protocolos de comunicação. Divergências no formato do payload JSON (substituição de campos booleanos por indicadores de status numéricos) causaram erros de servidor (HTTP 500), sendo solucionados através da padronização da estrutura de dados enviada pelo microcontrolador e recebida pela API, garantindo a robustez do fluxo de informações da borda para o banco de dados.

